UNIVERSITÉ HASSAN II DE CASABLANCA

École Nationale Supérieure d'Arts et Métiers de Casablanca

Département : Informatique

Filière : Intelligence Artificielle et Génie Informatique

Sujet :

CliniCare

Application de bureau de gestion d'une clinique

Réalisé par :

Fatima-Ezzahrae TOUIR

Israe FARISSY

Hiba RADID

Encadré par :

Pr. Mustapha HAIN

Année Universitaire 2026-2027

1. Introduction et contexte

1.1 Contexte

La gestion d'une clinique repose sur trois informations de base : les patients que l'on soigne, les médecins qui les soignent, et les rendez-vous qui les relient. Lorsque ces informations sont gérées sur papier ou dans des fichiers séparés, on risque des doublons, des oublis et des conflits d'horaires. Ce projet répond à ce besoin avec une application simple, utilisable par le personnel d'accueil.

1.2 Objectif

CliniCare est une application de bureau développée en Python avec la bibliothèque Tkinter pour l'interface graphique, et une base de données SQLite. Elle permet de gérer les patients, les médecins et les rendez-vous depuis une fenêtre unique, avec une navigation par boutons. Chaque module propose les quatre opérations de base : ajouter, afficher, modifier et supprimer, chacune étant enregistrée immédiatement dans la base de données.

1.3 Choix techniques

- **Python et Tkinter** : développement rapide d'une interface graphique, sans installation supplémentaire.
- **SQLite** : une base de données contenue dans un seul fichier, intégrée à Python, avec trois tables liées entre elles.
- **Git et GitHub** : travail en équipe, chacune développant son module sur sa propre branche.

1.4 Organisation du rapport

Ce rapport présente d'abord la conception de l'application (architecture et base de données), puis l'interface globale, puis chacun des trois modules avec ses règles de gestion et son fonctionnement. Il se termine par les tests, l'organisation du travail en équipe, et les perspectives d'amélioration.

2. Conception : architecture et base de données

2.1 Architecture de l'application

L'application est organisée en six fichiers Python, chacun avec un rôle précis. Cette séparation permet à chaque membre du groupe de travailler sur son module sans modifier celui des autres.

Fichier	Rôle
main.py	Fenêtre principale, menu de navigation et page d'accueil
style.py	Couleurs, polices et composants communs (bandeau, formulaire, boutons, tableau)
database.py	Connexion SQLite, création des 3 tables, fonctions d'accès à la base
patients.py	Module Patients (classe PatientsFrame)
medecins.py	Module Médecins (classe MedecinsFrame)

Fichier	Rôle
rendez_vous.py	Module Rendez-vous (classe RendezVousFrame)

Quatre principes guident cette architecture :

- Chaque module est une classe qui hérite de Frame : main.py peut donc les placer dans la même fenêtre et afficher celui que le menu désigne.
- Tous les modules construisent leur interface avec les composants de style.py, ce qui garantit la même apparence partout.
- Les trois modules ont la même structure de méthodes : creer_widgets, valider_champs, ajouter, afficher, selectionner, modifier, supprimer et vider_champs.

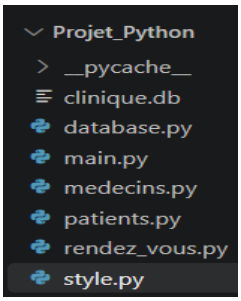


Figure 1 : Organisation des fichiers du projet

2.2 Base de données

L'application utilise SQLite : toutes les données sont stockées dans un seul fichier, clinique.db, créé automatiquement au premier lancement. Elle contient trois tables. La table rendez_vous relie un patient à un médecin : un patient peut avoir plusieurs rendez-vous, et un médecin aussi.

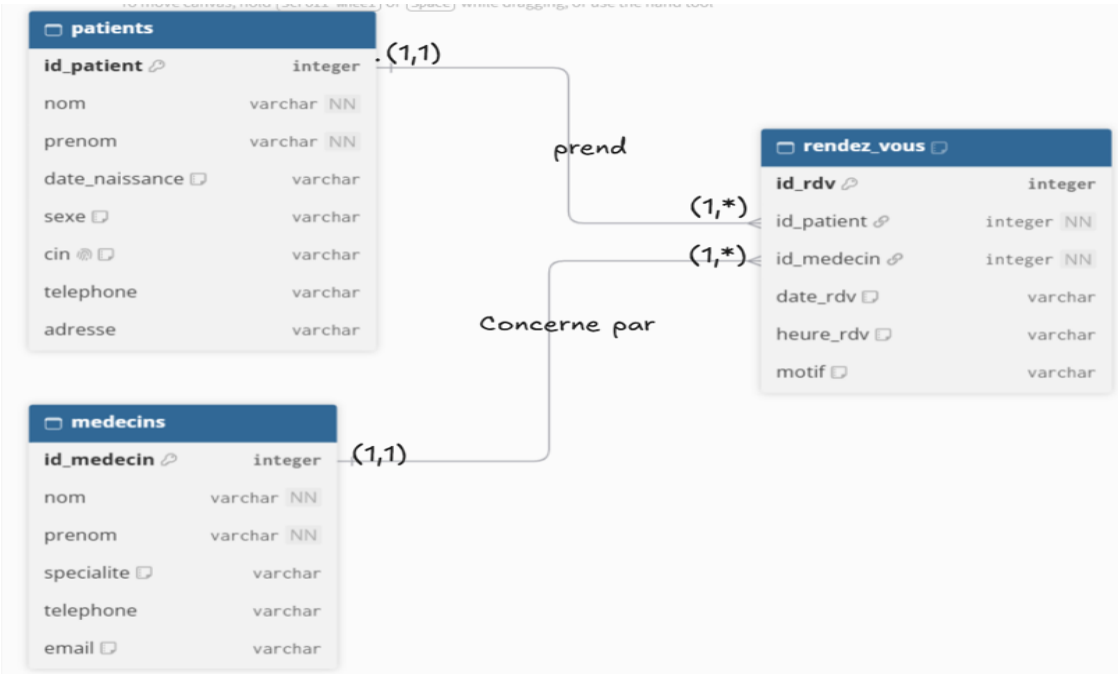


Figure 2 : Diagramme de classes des trois tables

Les règles de gestion sont inscrites dans la base elle-même, sous forme de contraintes :

Contrainte	Table	Effet
Clé primaire auto-incrémentée	les 3 tables	chaque ligne a un identifiant unique
CIN unique	Patients	impossible d'enregistrer deux fois le même patient
(nom, prénom, téléphone) unique	Medecins	pas de médecin en double
(médecin, date, heure) unique	rendez_vous	un médecin n'a pas deux rendez-vous au même moment
(patient, date, heure) unique	rendez_vous	un patient non plus
Clés étrangères	rendez_vous	on ne peut pas supprimer un patient ou un médecin qui a des rendez-vous

Ces règles sont vérifiées deux fois : dans l'interface, qui affiche un message clair à l'utilisateur, et dans la base, qui sert de dernière sécurité.

2.3 Fonctionnement général

Le déroulement d'une session d'utilisation est le suivant :

- Lancement : main.py crée les tables si elles n'existent pas, applique le style commun, construit les trois modules et affiche la page d'accueil.
- Navigation : l'utilisateur choisit un module avec le menu. À chaque changement de page, les données sont relues dans la base, ce qui garantit un affichage toujours à jour.
- Action : chaque ajout, modification ou suppression est envoyé immédiatement à la base, puis le tableau est actualisé.
- Ordre logique d'utilisation : on enregistre d'abord les médecins et les patients, puis on planifie les rendez-vous, qui ne peuvent s'appuyer que sur des patients et des médecins déjà enregistrés.

3. Interface globale

3.1 Fenêtre principale et navigation

La classe Application (fichier main.py) crée la fenêtre de l'application, applique le style commun, puis construit les trois modules et la page d'accueil. Un menu bleu en haut de la fenêtre propose cinq boutons : Accueil, Patients, Médecins, Rendez-vous et Quitter. Le bouton Quitter demande une confirmation avant de fermer l'application.

Les quatre pages sont empilées dans la même zone de la fenêtre. Un clic sur un bouton du menu amène la page choisie au premier plan. À chaque changement de page, celle-ci relit ses données dans la base : la page Rendez-vous recharge ainsi ses listes de patients et de médecins, et voit immédiatement ceux qui viennent d'être ajoutés.

3.2 Page d'accueil

La page d'accueil affiche le nom de la clinique, un message de bienvenue et trois compteurs : le nombre de patients, de médecins et de rendez-vous enregistrés. Ces compteurs sont recalculés à chaque affichage de la page.



Figure 3 : Page d'accueil et menu de navigation

4. Module Patients

Rôle : Le module Patients (patients.py, classe PatientsFrame) permet à la secrétaire d'enregistrer les nouveaux patients et de gérer les patients existants : ajouter, afficher, modifier, supprimer et rechercher.

Règles de gestion :

- Le nom, le prénom et le CIN sont obligatoires. Le CIN est enregistré en majuscules et ne peut exister qu'une seule fois.
- La date de naissance, si elle est saisie, doit être une date réelle au format JJ/MM/AAAA.
- Le téléphone, s'il est saisi, doit contenir exactement 10 chiffres.
- Le sexe est choisi dans une liste (M ou F).
- Un patient qui a des rendez-vous ne peut pas être supprimé. Toute suppression demande une confirmation.

Workflow : La secrétaire remplit le formulaire et clique sur Ajouter. Les champs sont validés : en cas d'erreur, un message précise le problème. Sinon le patient est enregistré, le tableau est actualisé et le formulaire est vidé. Pour modifier ou supprimer, elle clique sur une ligne du tableau : le formulaire se remplit avec les données du patient, relues dans la base. La zone Recherche filtre le tableau par nom ou par CIN.

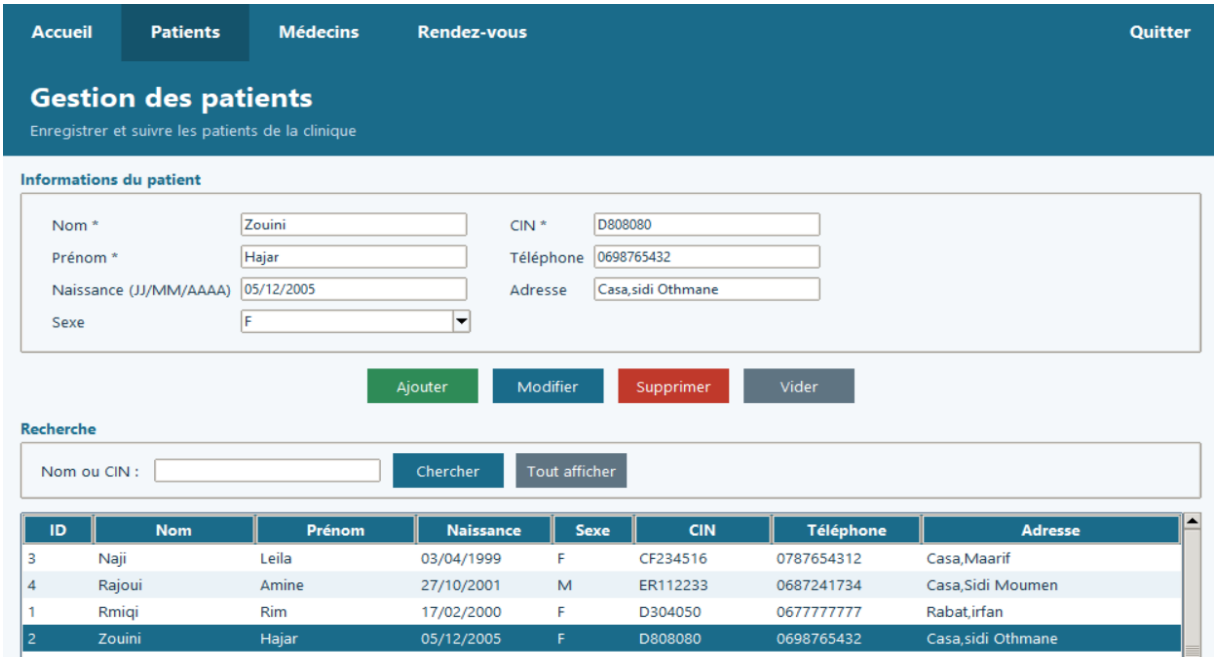


Figure 4 : Module Patients

5. Module Médecins

Rôle : Le module Médecins (medecins.py, classe MedecinsFrame) gère la liste des médecins de la clinique : ajouter, afficher, modifier et supprimer.

Règles de gestion :

- Le nom, le prénom et la spécialité sont obligatoires. La spécialité est choisie dans une liste (généraliste, cardiologue, pédiatre, dermatologue, gynécologue, ophtalmologue, dentiste, autre).
- Le téléphone, s'il est saisi, doit contenir 10 chiffres. L'email, s'il est saisi, doit contenir un @.
- Un même médecin (même nom, prénom et téléphone) ne peut pas être enregistré deux fois.
- Un médecin qui a des rendez-vous ne peut pas être supprimé.

Workflow : Il est identique à celui des patients : formulaire, validation, enregistrement, puis actualisation du tableau. Un clic sur une ligne recharge le médecin dans le formulaire pour le modifier ou le supprimer. Les médecins enregistrés ici apparaissent ensuite dans la liste de choix du module Rendez-vous, sous la forme « Nom Prénom (Spécialité) ».

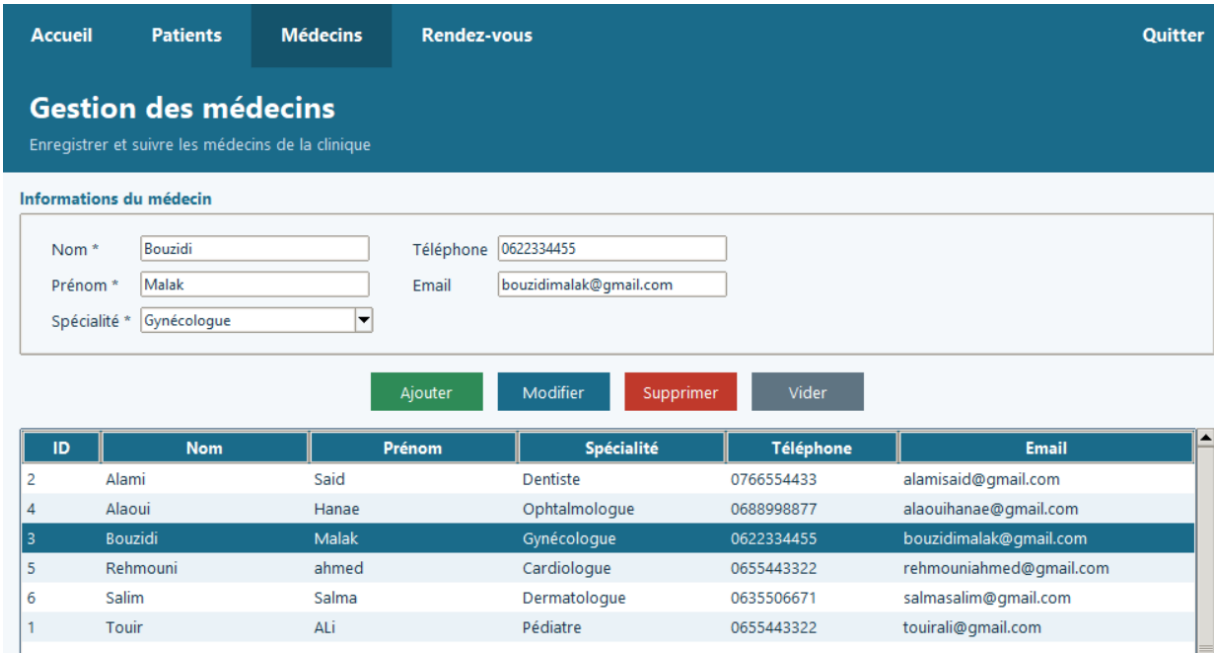


Figure 5 : Module Médecins

6. Module Rendez-vous

Rôle : Le module Rendez-vous (rendez_vous.py, classe RendezVousFrame) relie un patient à un médecin à une date et une heure données. C'est le module qui dépend des deux autres.

Règles de gestion :

- Le patient et le médecin se choisissent dans des listes alimentées par la base : on ne peut pas saisir un patient ou un médecin qui n'existe pas.
- La date doit être réelle (JJ/MM/AAAA). L'heure se choisit parmi des créneaux de 30 minutes, de 08:00 à 17:30. Le motif est facultatif.
- Un médecin ne peut pas avoir deux rendez-vous au même créneau, et un patient non plus.
- La suppression d'un rendez-vous demande une confirmation.

Workflow : À chaque ouverture du module, les listes de patients et de médecins sont rechargées, donc les nouveaux enregistrements sont disponibles immédiatement. L'utilisateur choisit un patient, un médecin, une date et une heure, puis clique sur Ajouter. Le programme vérifie d'abord la disponibilité du médecin et du patient ; si le créneau est libre, le rendez-vous est enregistré. Le tableau affiche les noms (et non les identifiants) grâce à une requête avec jointure, et il est trié par ordre chronologique.

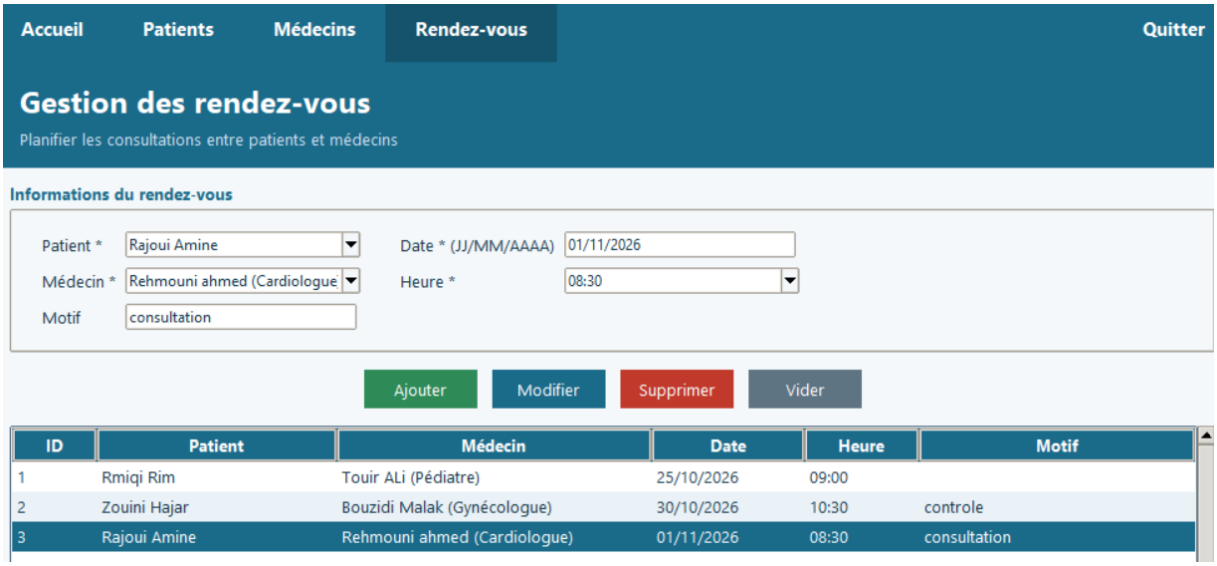


Figure 6 : Module Rendez-vous

La figure ci-dessous montre le cas où le patient a déjà un rendez-vous avec un médecin a une date et heure précise et veut ajouter un second rendez-vous à la même date et heure chez un autre médecin c'est le cas de Zouini Hajar qui veut prendre deux rendez-vous chez deux différents médecins le 30/10/2026 à 10:30, l'application affiche le message « Ce patient a déjà un rendez-vous a cette date et heure » et refuse l'ajout.

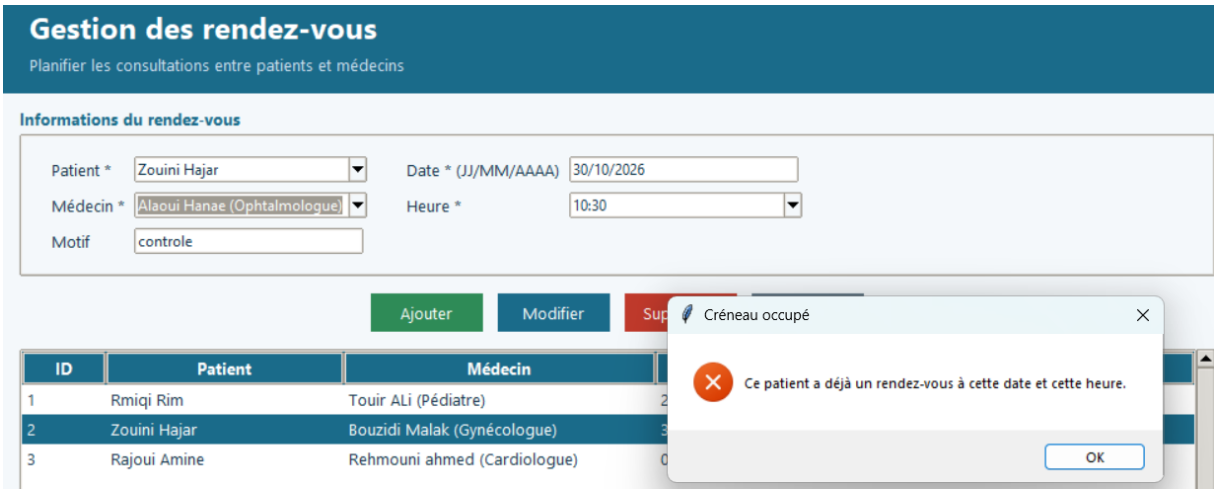


Figure 7 : Refus d'un rendez-vous en double cote patient

La figure ci-dessous montre le cas où le patient veux prendre un rendez-vous à une date et heure précise chez un médecin précis mais ce médecin a déjà un autre rendez-vous dans ce même créneau et donc, l'application affiche le message « Ce médecin a déjà un rendez-vous à cette date et heure » et refuse l'ajout.

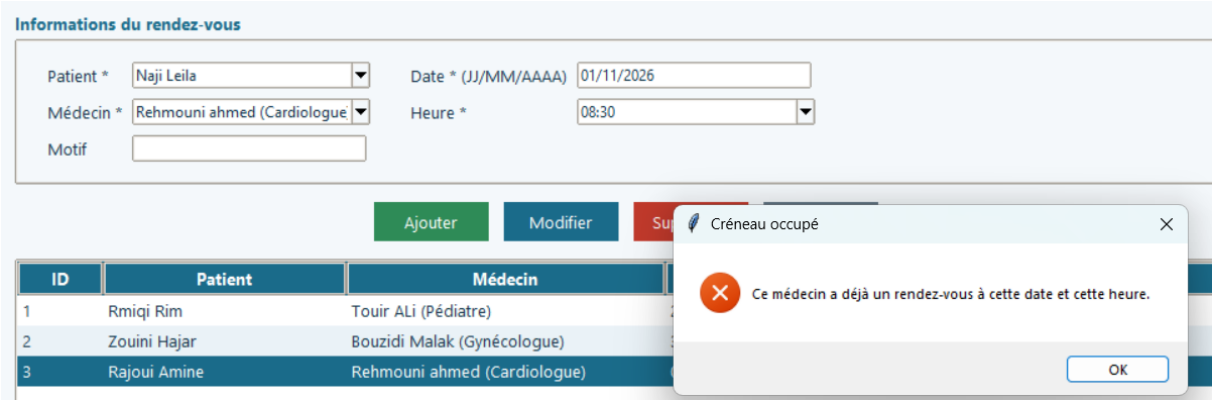


Figure 8 : Refus d'un rendez-vous en double cote medecin

7. Tests, difficultés et perspectives

Tests : L'application a été testée sur un scénario complet (ajout d'un médecin, d'un patient, puis d'un rendez-vous) et sur des cas d'erreur volontaires :

Test	Résultat
Ajouter deux fois le même CIN	Refusé : « Ce CIN existe déjà »
Ajouter deux fois le même médecin	Refusé : « Ce médecin existe déjà »
Deux rendez-vous du même médecin au même créneau	Refusé
Deux rendez-vous du même patient au même créneau	Refusé
Supprimer un patient ou un médecin qui a des rendez-vous	Refusé avec un message
Saisir une date inexistante (31/02/2026)	Refusé

8. Difficultés rencontrées.

- Un même médecin pouvait être ajouté plusieurs fois, et un patient pouvait avoir deux rendez-vous simultanés : nous avons ajouté des contraintes dans la base, doublées de messages clairs dans l'interface.
- Les trois interfaces n'avaient pas la même apparence : nous avons créé un fichier de style commun à tous les modules.

9. Conclusion et perspectives

CliniCare répond au besoin d'origine : une application unique, simple à utiliser, qui gère les patients, les médecins et les rendez-vous avec un CRUD complet relié en temps réel à une base SQLite. Les règles de gestion garantissent la cohérence des données (pas de doublons, pas de conflits d'horaires), et les trois modules partagent la même apparence et la même structure de code. Ce projet nous a aussi permis de pratiquer le travail collaboratif avec Git et GitHub.

Plusieurs améliorations sont possibles : une authentification des utilisateurs (secrétaire, médecin), des recherches et des filtres sur les médecins et les rendez-vous, un agenda par médecin et par jour, et l'export des rendez-vous en PDF ou en Excel.